

SMART EYEGLOSS WITH OBSTACLE DETECTION & FACIAL RECOGNITION

*A Report submitted
in partial fulfillment for the Degree of*
Bachelor of Technology

**in
Computer Engineering**

**by
MD MINHAZ (19BCS014)
SHAHABUDDIN (19BCS031)**

Under the Supervision of
PROF MOHAMMAD AMJAD



**Department of Computer Engineering
F/O Engineering & Technology
Jamia Millia Islamia New Delhi -110025**

May, 2023

CERTIFICATE

This is to certify that the project report entitled *Smart eyeglass with obstacle detection & facial recognition* submitted by Md Minhaaz (19BCS014) and Shahabuddin (19BCS031) to the Department of Computer Engineering, F/O Engineering & Technology, Jamia Millia Islamia New Delhi – 110025 in partial fulfilment for the award of the degree of B. Tech in (Computer Engineering) is a bona fide record of project work carried out by him/her under my/our supervision. The contents of this report, in full or in parts, have not been submitted to any other Institution or University for the award of any degree.

Prof. Mohammad Amjad

Supervisor

Department of Computer Engineering

Prof. Bashir Alam

Head of Department

Department of Computer Engineering

New Delhi

May, 2023

DECLARATION

I declare that this project report titled *Smart eyeglass with obstacle detection & facial recognition* submitted in partial fulfilment of the degree of B. Tech in (Computer Engineering) is a record of original work carried out by me under the supervision of **Prof. Mohammad Amjad**, and has not formed the basis for the award of any other degree, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Md Minhaaz
19BCS014

Shahabuddin
19BCS031

New Delhi-110025
29 May, 2023

ACKNOWLEDGEMENT

We would like to convey our heartfelt gratitude to **Prof. Mohammad Amjad**, our mentor, for his invaluable advice and assistance in completing our project. He was there to assist us every step of the way, and his motivation is what enabled us to accomplish our task effectively. We would also like to thank all of the other supporting personnel who assisted us by giving us the time that was essential and vital, without which we would not have been able to perform efficiently on this project. We would also like to thank our institute Jamia Millia Islamia for accepting our project. We would also like to thank our friends and parents for their support and encouragement as we worked on this assignment without whose love and support, we would have not been here today

Md Minhaaz (19BCS014)

Shahabuddin (19BCS031)

ABSTRACT

Visually impaired and blind people require constant assistance from others to move around, making them extremely dependent on others in their social lives. For visually impaired and blind people, object detection and distance calculation are major challenges. Earlier navigation systems were both costly and time-consuming to use in everyday life. This study describes a blind or visually impaired person's electronic navigation system (subject). Using an ultrasonic sensor network, this system detects objects. It effectively measures the distance between the person and the detected obstacle(object). It employs speech feedback to inform the user of the location and distance of the indicated obstacle. Speaker is used to provide such voice messages to the subject. It also uses camera and facial recognition algorithms to detect faces and recognize the person and inform the user through audio output. The project's objective is to equip visually impaired and blind people with a more cost-effective and portable navigation system.

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
CERTIFICATE	i
DECLARATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF FIGURES	viii
ABBREVIATIONS	ix
1. INTRODUCTION	1
1.1 Introduction	1
1.2 Objective	1
2. AIOT	2
2.1 Introduction	2
2.2 AI	4
2.3 IOT	6
3. TOOLS & TECHNOLOGY	8
3.1 Python	8
3.2 OpenCV	10
3.3 Face Recognition	11
3.4 eSpeak	13

3.5 Raspberry Pi	14
3.6 Ultrasonic Sensor	16
3.7 Camera	19
3.8 Eyeglass	20
3.9 Wires	21
3.10 Battery	21
3.11 Earphone	22
4. CONNECTIONS & CODE	23
4.1 Ultrasonic Sensor	23
4.2 Raspberry Pi Camera	25
4.3 Full Connection	29
5. WORKING	34
5.1 Steps	34
6. RESULT	35
7. LIMITATIONS	36
8. SOLUTIONS OF LIMITATIONS	37
REFERENCES	38

LIST OF FIGURES

FIGURE	TITLE	PAGE NUMBER
1.1	Objective	1
2.1	AIot	3
2.2	AI	5
2.3	IoT	7
3.1	Extracting faces	11
3.2	Extracting features	12
3.3	Identifying faces	13
3.4	Raspberry Pi	16
3.5	Ultrasonic sensor	17
3.6	Working Principle	18
3.7	Distance Calculation	18
3.8	Raspberry Pi Camera	20
3.9	Eyeglass	20
3.10	Wires	22
3.11	Battery	21
3.12	Earphone	22
4.1	Block Diagram of Connection	23
4.2	Ultrasonic sensor connected to Raspberry Pi	24
4.3	Camera connected to Raspberry Pi	26
4.4	Full connection on Eyeglass	29
5.1	Work Flow	34
7.1	Limitations of Ultrasonic sensor	36

ABBREVIATIONS

AI	Artificial Intelligence
AIOT	Artificial Intelligence of Things
IoT	Internet of Things

CHAPTER 1

INTRODUCTION

1.1 Introduction

Globally, there are 2.2 billion people who are visually impaired or blind, according to 2022 World Health Organization research on disability. As the population is growing older and as people's habits change, chronic eye diseases like diabetic retinopathy are likely to become more common. Eyes are the most important organ to perceive the outside world. Without eyes it is difficult to move. People who are blind or visually impaired mostly rely on other people to move around. Earlier it was difficult to create a sophisticated navigation system for such people because it was too costly and time consuming. Moreover, most of the sensors were of huge sizes, which made it difficult to use it on basis. But due to advancements in technology in recent times, now sensors have become cheaper, and their size has also been reduced. It is now possible to create a eyeglass using ultrasonic sensors and ML, which will help visually impaired people to move around without being dependent on other people. As soon as an obstacle comes in way, the glasses will alert the user using a voice, so that user can know that there is an obstacle at some distance and user can avoid it. We'll also include facial recognition, so that if a known person comes in front of a person, that person can easily recognize another person.

1.2 Objective

The main objective of this project is to enable differently abled persons to navigate easily. It will solve the problem of obstacle and facial recognition. People will be able to walk easily without colliding with obstacles. It can also be used for facial recognition using the camera which is attached on the glass.

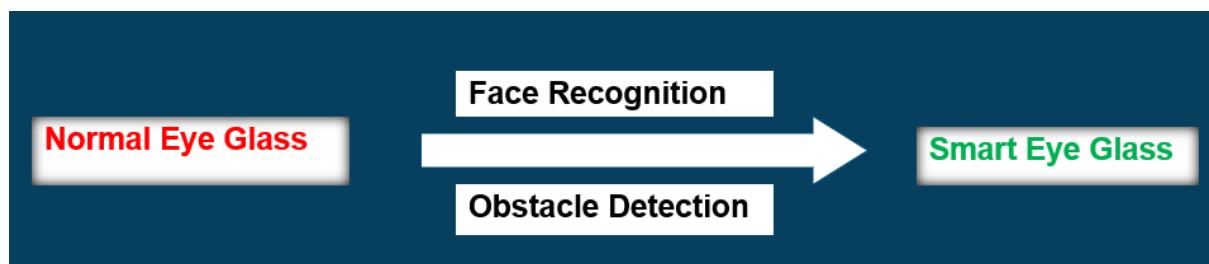


Fig 1.1: Objective

CHAPTER 2

AIOT

2.1 Introduction

Artificial Intelligence of Things (AIoT) is the combination of artificial intelligence (AI) and the internet of things (IoT). AI is the ability of machines to learn and make decisions without human intervention. IoT is the network of physical devices that are connected to the internet and can collect and exchange data.

AIoT brings together the strengths of AI and IoT to create new and innovative applications. For example, AI can be used to analyze data collected by IoT devices to identify patterns and trends that would be difficult or impossible to see with the naked eye. This information can then be used to make better decisions, such as optimizing production processes or predicting customer behavior.

AIoT is still in its early stages of development, but it has the potential to revolutionize many industries. Some of the most promising AIoT applications include:

- **Smart cities:** AIoT can be used to make cities more efficient and sustainable. For example, AI can be used to control traffic lights, optimize energy use, and manage waste disposal.
- **Healthcare:** AIoT can be used to improve patient care and reduce costs. For example, AI can be used to monitor patients remotely, diagnose diseases earlier, and personalize treatment plans.
- **Manufacturing:** AIoT can be used to improve production efficiency and quality. For example, AI can be used to identify defects in products, optimize production schedules, and manage inventory levels.
- **Retail:** AIoT can be used to improve customer service and personalization. For example, AI can be used to recommend products to customers, track customer behavior, and personalize marketing campaigns.

AIoT is a powerful new technology with the potential to change the world. As it continues to develop, we can expect to see even more innovative and groundbreaking applications.

Here are some additional benefits of AIoT:

- **Increased efficiency:** AIoT can help businesses to operate more efficiently by automating tasks and processes. This can lead to cost savings and improved productivity.
- **Improved decision-making:** AIoT can help businesses to make better decisions by providing them with insights into their data. This can help them to identify trends, optimize operations, and improve customer service.
- **Enhanced customer experience:** AIoT can help businesses to provide their customers with a more personalized and engaging experience. This can lead to increased customer loyalty and satisfaction.

Overall, AIoT is a powerful new technology with the potential to revolutionize many industries. As it continues to develop, we can expect to see even more innovative and groundbreaking applications.

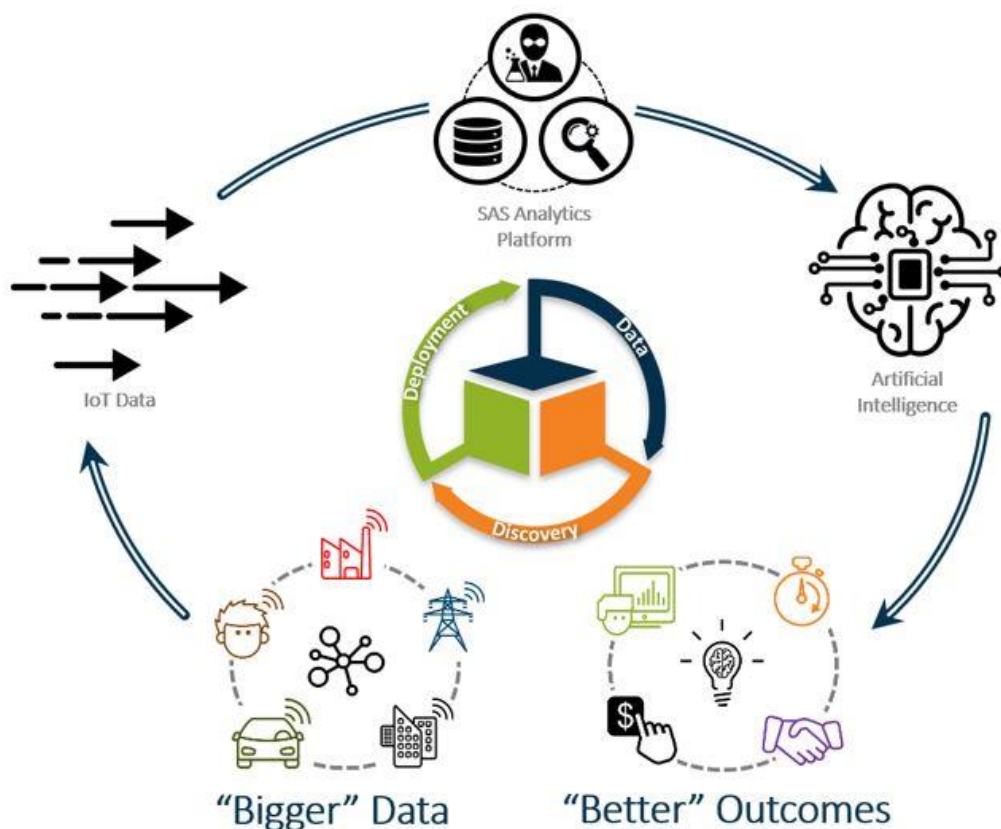


Fig 2.1: AIoT

2.2 AI

Artificial intelligence (AI) is a branch of computer science that deals with the creation of intelligent agents, which are systems that can reason, learn, and act autonomously. AI research has been highly successful in developing effective techniques for solving a wide range of problems, from game playing to medical diagnosis.

There are many different approaches to AI, but one of the most popular is machine learning. Machine learning algorithms are trained on large amounts of data, and they can then use this data to make predictions or decisions. For example, a machine learning algorithm could be trained on a dataset of medical images, and it could then be used to diagnose diseases.

Another important area of AI research is natural language processing (NLP). NLP is concerned with the ability of machines to understand and generate human language. NLP techniques are used in a wide variety of applications, such as speech recognition, machine translation, and text summarization.

AI is a rapidly growing field, and it is having a major impact on many different industries. AI is being used to develop new products and services, to improve existing processes, and to make better decisions. As AI continues to develop, it is likely to have an even greater impact on our lives.

Here are some of the most common domains of AI:

- **Machine learning:** Machine learning is a type of AI that allows computers to learn without being explicitly programmed. This is done by feeding the computer large amounts of data and allowing it to identify patterns in the data. Once the computer has identified these patterns, it can use them to make predictions about new data.
- **Deep learning:** Deep learning is a type of machine learning that uses artificial neural networks to learn from data. Artificial neural networks are inspired by the human brain, and they are able to learn complex patterns from data that would be difficult for traditional machine learning algorithms to learn.

- **Natural language processing:** Natural language processing (NLP) is a field of computer science that deals with the interaction between computers and human (natural) languages. NLP is used in a wide variety of applications, such as speech recognition, machine translation, and text summarization.
- **Computer vision:** Computer vision is a field of computer science that deals with the extraction of meaning from digital images or videos. Computer vision is used in a wide variety of applications, such as facial recognition, object detection, and scene understanding.
- **Robotics:** Robotics is a field of engineering that deals with the design, construction, operation, and application of robots. Robots are used in a wide variety of industries, such as manufacturing, healthcare, and agriculture.

These are just a few of the many domains of AI. As AI continues to develop, it is likely that new domains will emerge.

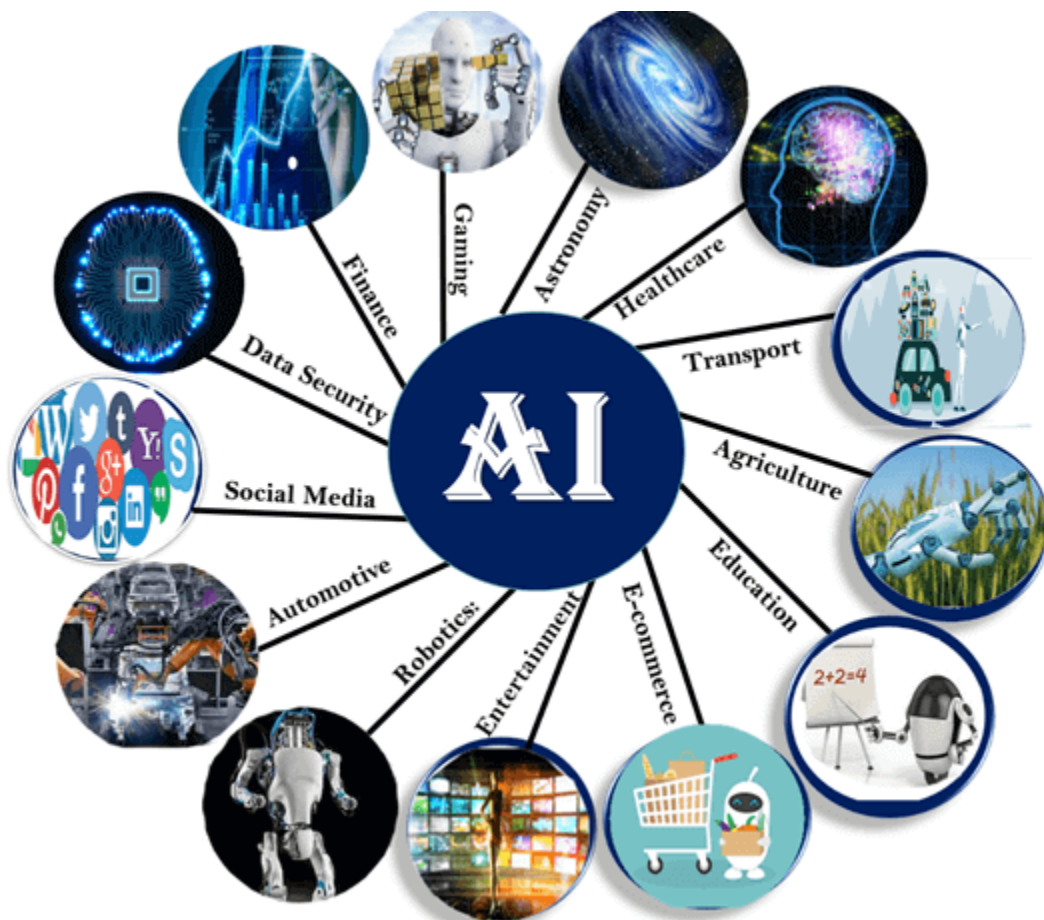


Fig 2.2: AI

2.3 IOT

IoT stands for the Internet of Things. It refers to a network of physical objects, devices, vehicles, appliances, and other items embedded with sensors, software, and connectivity, enabling them to collect and exchange data over the internet. These connected devices are capable of gathering information, communicating with each other and with humans, and performing various tasks to enhance efficiency, convenience, and functionality in our daily lives.

The concept behind IoT is to extend the capabilities of traditional computing devices, such as computers and smartphones, to a vast range of everyday objects. By equipping objects with sensors and internet connectivity, IoT enables them to interact and share data without human intervention. This connectivity allows for intelligent and automated decision-making, creating a seamless integration of the physical and digital worlds.

The key components of an IoT system are:

- **Devices:** Physical objects or "things" that are equipped with sensors, actuators, and connectivity modules. These devices can be as small as a wearable fitness tracker or as large as an industrial machine.
- **Sensors and Actuators:** Sensors capture data from the environment, such as temperature, humidity, light, motion, or pressure. Actuators are used to perform actions based on the data received, such as turning on a light, adjusting the temperature, or opening a door.
- **Connectivity:** IoT devices use various communication protocols like Wi-Fi, Bluetooth, Zigbee, or cellular networks to connect to the internet and share data with other devices or cloud platforms.
- **Data Processing:** IoT generates a massive amount of data, and processing this data is crucial for extracting meaningful insights. Data processing can occur on the device itself, on a local gateway, or in the cloud, depending on the complexity and requirements of the application.
- **Cloud Services:** IoT devices often leverage cloud-based platforms for storage, data analysis, and computing power. Cloud services offer scalability, real-time analytics, and accessibility from anywhere.

- **Applications and Analytics:** IoT data can be analysed to gain insights, identify patterns, and make informed decisions. This information can be used to optimize operations, improve efficiency, and create new services and applications.

IoT has a wide range of applications across various industries, including smart homes, healthcare, transportation, agriculture, manufacturing, energy management, and more. It has the potential to revolutionize how we live and work by enabling smarter and more connected environments, enhancing automation, and enabling new levels of efficiency and convenience. However, it also raises concerns related to privacy, security, and the ethical use of data, which need to be addressed as IoT continues to evolve.

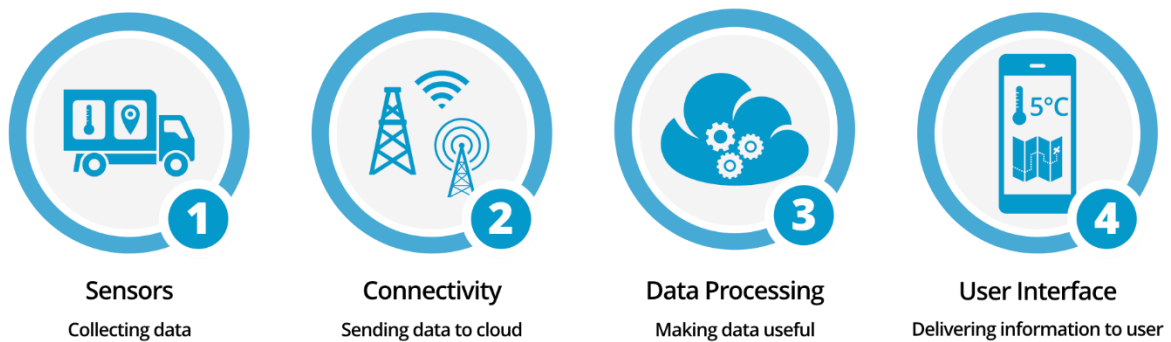


Fig 2.3: IoT

Chapter 3

TOOLS & TECHNOLOGY

In this project, we used the following technology and tools.

- Python
- OpenCV
- Face Recognition
- eSpeak
- Raspberrypi
- Ultrasonic Sensor
- Camera
- Eyeglass
- Wires
- Battery
- Earphone

3.1 Python

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. Created by Guido van Rossum and first released in 1991, Python has since gained widespread popularity and has become one of the most commonly used programming languages. Here is a detailed overview of Python's key features and characteristics:

- **Readability:** Python emphasizes code readability and uses a clean and consistent syntax, making it easy to understand and write. The use of whitespace indentation instead of curly braces or keywords enhances the readability and structure of the code.
- **Simplicity:** Python strives for simplicity and minimalistic design. It provides a small set of core concepts and features, making it easier for beginners to grasp and write code quickly.
- **Versatility:** Python supports both object-oriented programming (OOP) and procedural programming paradigms. It offers a wide range of libraries and frameworks, making it

suitable for various domains, including web development, scientific computing, data analysis, artificial intelligence, and more.

- **Interpreted:** Python is an interpreted language, meaning that the code is executed line by line at runtime without the need for explicit compilation. This feature enables faster development and easier debugging.
- **Dynamically Typed:** Python is dynamically typed, which means variable types are inferred at runtime. You don't need to explicitly declare variable types, making it flexible and allowing for quicker development. However, this can sometimes lead to potential type-related errors.
- **Large Standard Library:** Python comes with a comprehensive standard library that provides a rich set of modules and functions for various purposes, such as file I/O, networking, string manipulation, regular expressions, and more. This extensive library reduces the need for external dependencies in many cases.
- **Third-Party Libraries and Ecosystem:** Python has a vast and active community that has developed numerous third-party libraries and frameworks, extending its capabilities for specific use cases. Some popular libraries include NumPy for numerical computing, Pandas for data manipulation, TensorFlow for machine learning, Django for web development, Flask for building web applications, and many more.
- **Cross-Platform:** Python is available on multiple platforms, including Windows, macOS, Linux, and various other operating systems. This cross-platform compatibility allows developers to write code on one system and run it on another without major modifications.
- **Rapid Prototyping:** Python's simplicity and readability make it an excellent choice for rapid prototyping and quick development cycles. Its extensive libraries and frameworks facilitate faster development of projects.
- **Community and Documentation:** Python has a large and supportive community, which contributes to the language's growth and evolution. There are numerous online resources, tutorials, and documentation available, making it easier to learn and solve problems.

Python has gained significant popularity in fields like web development, data science, machine learning, and automation. It is often the language of choice for beginners due to its gentle learning curve and extensive resources. Whether you're a beginner or an experienced developer, Python provides a flexible and powerful platform for a wide range of applications.

3.2 OpenCV

OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning software library. It provides a wide range of tools, algorithms, and functions for image and video processing, as well as for computer vision tasks. OpenCV is widely used in various fields such as robotics, augmented reality, surveillance, medical imaging, and more.

Here are some key aspects of OpenCV:

- **Image and Video Input/Output:** OpenCV allows you to read, write, and manipulate images and videos from various sources such as files, cameras, or network streams.
- **Image Processing:** OpenCV offers a rich set of image processing functions, including filtering, morphological operations, thresholding, histogram equalization, color space conversions, and geometric transformations. These operations enable tasks such as noise removal, edge detection, image enhancement, and geometric correction.
- **Feature Detection and Description:** OpenCV provides algorithms for detecting and describing local image features, such as corners, blobs, and edges. These features are useful for tasks like image matching, object recognition, and tracking.
- **Machine Learning and Object Detection:** OpenCV integrates with machine learning libraries like TensorFlow and PyTorch, enabling the development and deployment of computer vision models. It includes pre-trained models for object detection, face recognition, and more. OpenCV's machine learning functionalities also support training custom models on user-provided datasets.
- **Camera Calibration and 3D Vision:** OpenCV includes tools for camera calibration, stereo vision, and 3D reconstruction. These features are crucial for tasks like depth estimation, 3D object modelling, and camera pose estimation.
- **User Interface and GUI:** OpenCV provides graphical user interface (GUI) functions to create windows, handle user events, and display images or videos. It simplifies the process of building interactive applications for computer vision tasks.
- **Cross-Platform and Language Support:** OpenCV is a cross-platform library, meaning it can be used on different operating systems such as Windows, macOS, Linux, and even mobile platforms like Android and iOS. It offers interfaces for popular programming languages like C++, Python, and Java, making it accessible to a broad range of developers.

OpenCV has a large and active community that continuously contributes to its development, ensuring that new features, bug fixes, and improvements are regularly released. Its versatility, performance, and ease of use make it a go-to choice for many computer vision and image processing projects.

3.3 Face Recognition

We used face-recognition module of python for facial-recognition. Built using dlib's state-of-the-art face recognition built with deep learning. The model has an accuracy of 99.38% on the Labeled Faces in the Wild benchmark.

This also provides a simple `face_recognition` command line tool that lets you do face recognition on a folder of images from the command line.

It has following features:

- **Find faces in pictures:** Find all the faces that appear in a picture.

```
• import face_recognition
• image = face_recognition.load_image_file("your_file.jpg")
• face_locations = face_recognition.face_locations(image)
```

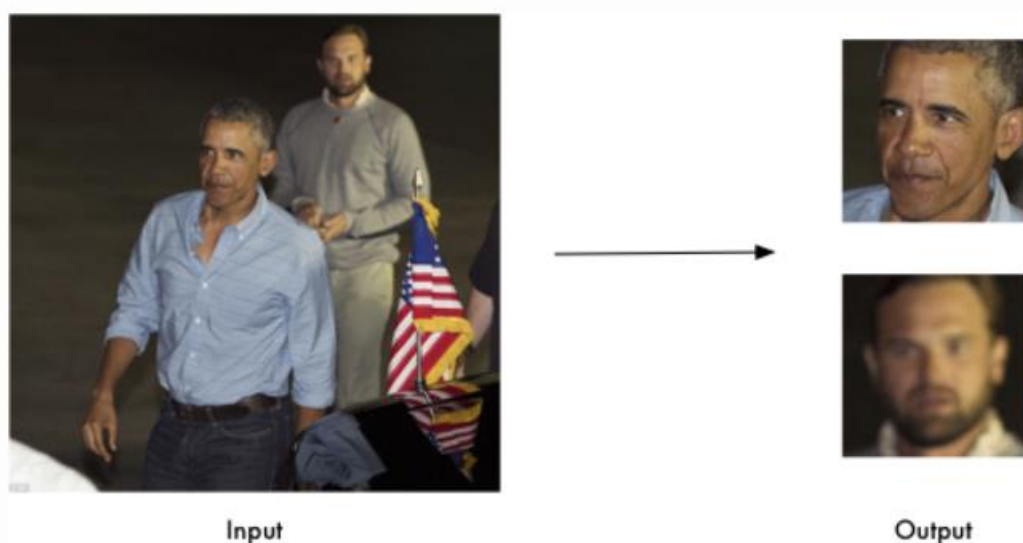


Fig 3.1: Extracting faces

- **Find and manipulate facial features in pictures:** Get the locations and outlines of each person's eyes, nose, mouth and chin. Finding facial features is super useful for lots of important stuff. But you can also use for really stupid stuff like applying digital make-up.

```
import face_recognition
image = face_recognition.load_image_file("your_file.jpg")
face_landmarks_list = face_recognition.face_landmarks(image)
```

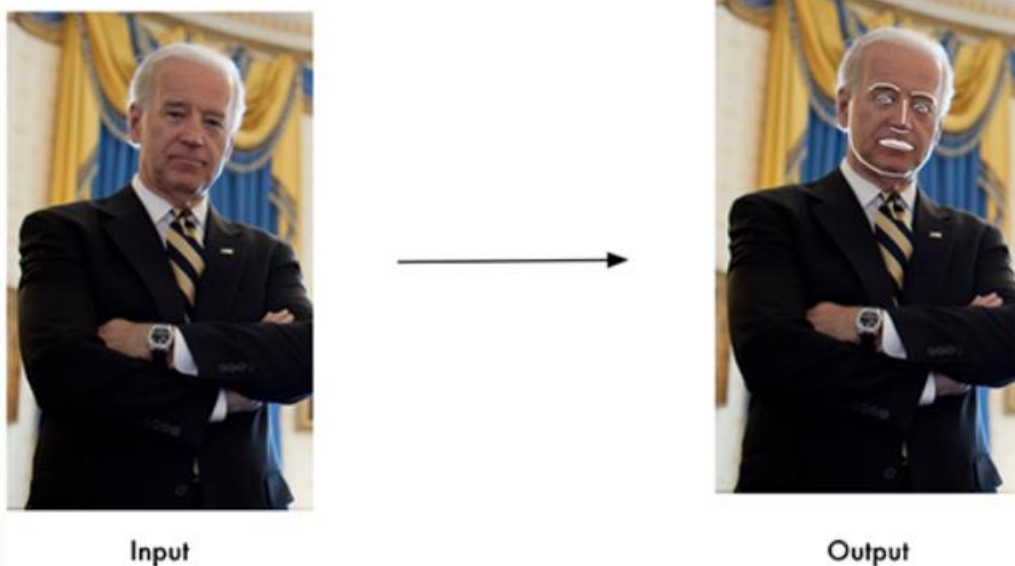


Fig 3.2: Extracting features

- **Identify faces in pictures:** Recognize who appears in each photo.

```
• import face_recognition
• known_image = face_recognition.load_image_file("biden.jpg")
• unknown_image = face_recognition.load_image_file("unknown.jpg")
•
• biden_encoding = face_recognition.face_encodings(known_image)[0]
• unknown_encoding = face_recognition.face_encodings(unknown_image)[0]
•
• results = face_recognition.compare_faces([biden_encoding],
    unknown_encoding)
```

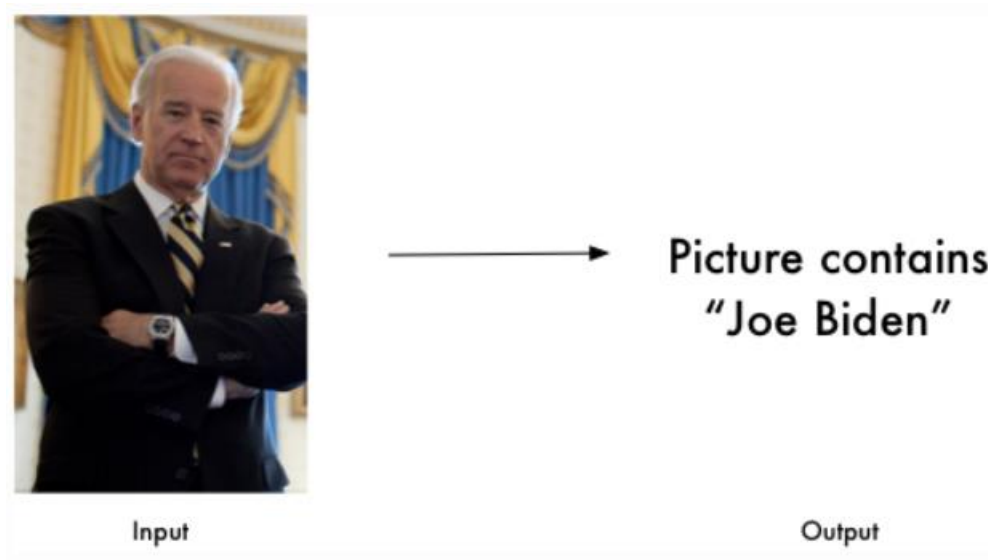


Fig 3.3: Identifying faces

3.4 eSpeak

eSpeak is a compact open source software speech synthesizer for English and other languages, for Linux and Windows. eSpeak uses a "formant synthesis" method. This allows many languages to be provided in a small size. The speech is clear, and can be used at high speeds, but is not as natural or smooth as larger synthesizers which are based on human speech recordings.

Features of eSpeak:

- Includes different Voices, whose characteristics can be altered.
- Can produce speech output as a WAV file.
- SSML (Speech Synthesis Markup Language) is supported (not complete), and also HTML.
- Compact size. The program and its data, including many languages, totals about 2 Mbytes.

- Can be used as a front-end to MBROLA diphone voices. eSpeak converts text to phonemes with pitch and length information.
- Can translate text into phoneme codes, so it could be adapted as a front end for another speech synthesis engine.
- Potential for other languages. Several are included in varying stages of progress. Help from native speakers for these or other languages is welcome.
- Development tools are available for producing and tuning phoneme data.
- Written in C.

Languages. The eSpeak speech synthesizer supports several languages, however in many cases these are initial drafts and need more work to improve them.

eSpeak does text to speech synthesis for the following languages, some better than others. It can speak the following languages:

Afrikaans, Albanian, Aragonese, Armenian, Bulgarian, Cantonese, Catalan, Croatian, Czech, Danish, Dutch, English, Esperanto, Estonian, Farsi, Finnish, French, Georgian, German, Greek, Hindi, Hungarian, Icelandic, Indonesian, Irish, Italian, Kannada, Kurdish, Latvian, Lithuanian, Lojban, Macedonian, Malaysian, Malayalam, Mandarin, Nepalese, Norwegian, Polish, Portuguese, Punjabi, Romanian, Russian, Serbian, Slovak, Spanish, Swahili, Swedish, Tamil, Turkish, Vietnamese, Welsh.

3.5 Raspberry Pi

For our project, we used **Raspberry Pi 3**.

Raspberry Pi is a series of single-board computers (SBCs) developed by the Raspberry Pi Foundation, a UK-based charity organization. The Raspberry Pi boards are designed to be low-cost, credit card-sized computers that provide a platform for learning programming, electronics, and computing.

Here are some key features and aspects of Raspberry Pi:

- **Hardware:** Raspberry Pi boards consist of a system-on-a-chip (SoC) that integrates a processor, memory, and other essential components onto a single board. The most recent models feature powerful ARM-based processors, varying amounts of RAM, USB ports, HDMI output for connecting to displays, Ethernet or Wi-Fi connectivity, and GPIO (General Purpose Input/Output) pins for interfacing with external electronics.
- **Versatility:** Raspberry Pi can be used for a wide range of projects, including educational purposes, hobbyist projects, home automation, media centers, retro gaming consoles, robotics, and more. Its small form factor and low power consumption make it suitable for various applications.
- **Operating System:** Raspberry Pi supports different operating systems, with the most popular being Raspberry Pi OS (formerly known as Raspbian), a Linux-based distribution optimized for the Raspberry Pi. However, other operating systems like Ubuntu, Windows 10 IoT Core, and various specialized distributions are also available.
- **GPIO Interface:** One of the distinguishing features of Raspberry Pi is its GPIO header, which allows users to connect and control external electronic components, such as sensors, motors, LEDs, and more. This feature makes it a valuable tool for learning and prototyping in the field of electronics and physical computing.
- **Programming and Software Development:** Raspberry Pi supports a variety of programming languages, including Python, C/C++, Java, and more. It provides an ideal environment for learning coding and software development. Additionally, various software libraries and tools, including the popular Python library RPi.GPIO, are available to interact with the GPIO interface.
- **Community and Resources:** Raspberry Pi has a vibrant and supportive community, consisting of enthusiasts, educators, and professionals. The community actively shares projects, tutorials, and resources, making it easy to find support and inspiration for your own Raspberry Pi endeavors.

Due to its affordability, versatility, and accessibility, Raspberry Pi has gained significant popularity worldwide, especially in education and the maker community. It has become a powerful tool for learning, prototyping, and building innovative projects that leverage computing and electronics.

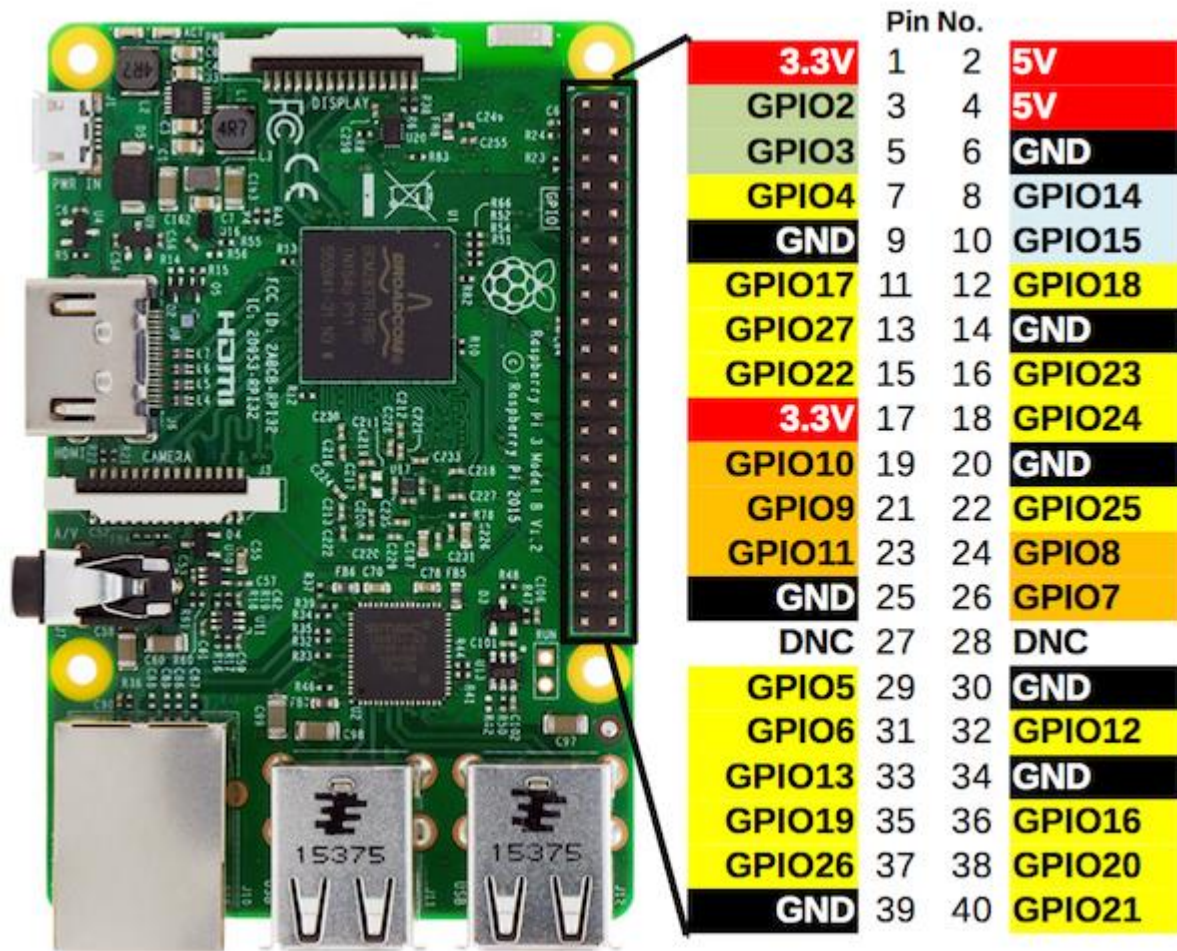


Fig 3.4: Raspberry Pi

3.6 Ultrasonic Sensor

An ultrasonic sensor is a device that uses sound waves at frequencies above the range of human hearing (typically above 20 kHz) to measure distances and detect objects. It works based on the principle of echolocation, similar to how bats navigate and detect obstacles in their path.

Here's how an ultrasonic sensor typically operates:

- **Emitting Sound Waves:** The sensor emits a high-frequency sound wave, usually in the ultrasonic range, using a piezoelectric transducer. The transducer converts electrical energy into mechanical vibrations, producing sound waves.
- **Reflection of Sound Waves:** The emitted sound waves propagate through the air and when they encounter an object in their path, they reflect off the object's surface.

- **Receiving Reflected Waves:** The sensor has a receiver component that detects the reflected sound waves. The receiver also consists of a piezoelectric transducer that converts the mechanical vibrations caused by the reflected waves into electrical signals.
- **Time Measurement:** The sensor measures the time it takes for the sound wave to travel from the sensor to the object and back. Since the speed of sound is known, the time measurement can be used to calculate the distance to the object.
- **Distance Calculation:** By knowing the speed of sound in the medium (usually air), the sensor can calculate the distance to the object using the formula: **Distance = (Speed of Sound × Time) / 2**. Dividing the time by 2 is necessary because the sound wave travels to the object and back.

Ultrasonic sensors are commonly used in various applications, including robotics, distance measurement, object detection, collision avoidance systems, parking sensors, and industrial automation. They offer advantages such as non-contact measurement, wide detection range, and relatively low cost. Additionally, some ultrasonic sensors may have additional features like adjustable sensitivity and beam width to adapt to specific application requirements.

It's important to note that the performance and accuracy of ultrasonic sensors can be influenced by environmental factors such as temperature, humidity, and the nature of the objects being detected (e.g., shape, material, surface properties).

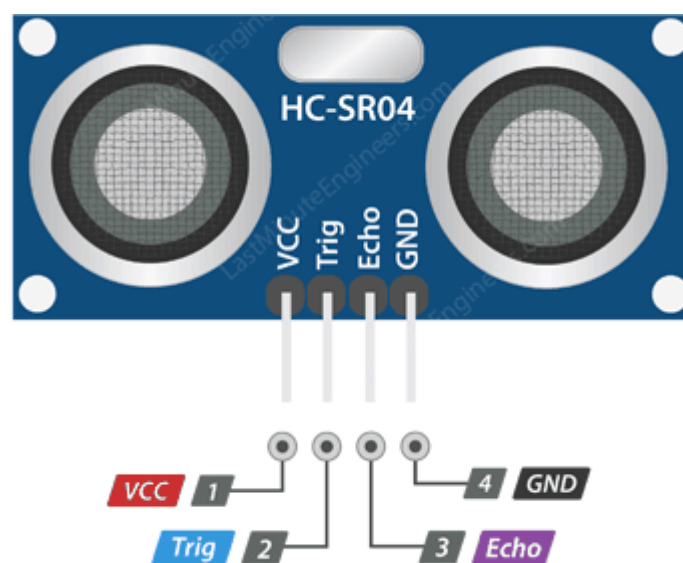
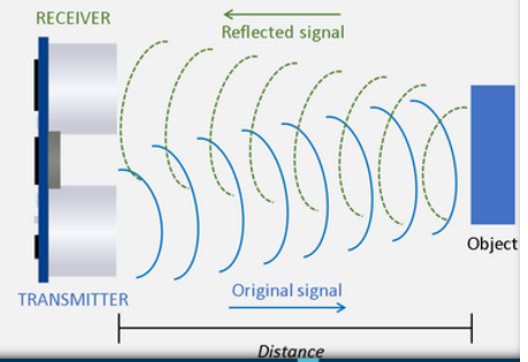
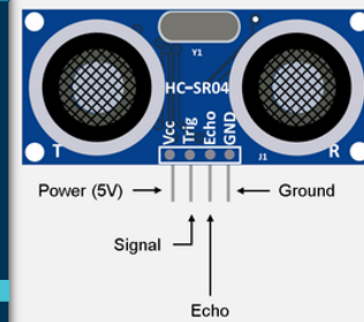


Fig 3.5: Ultrasonic sensor

The HC-SR04 Ultrasonic sensor has four pins.

1. Vcc
2. TRIG (input)
3. ECHO (output)
4. GND



- Here, *time taken* = transmitter to object + object to receiver
- Thus, we need to consider *half of the time taken* (time taken/2) to calculate distance.
- Finally, *distance* = speed of sound $\times$ (time taken/2)

$$\text{distance} = \frac{\text{speed of sound} \times \text{time taken}}{2}$$

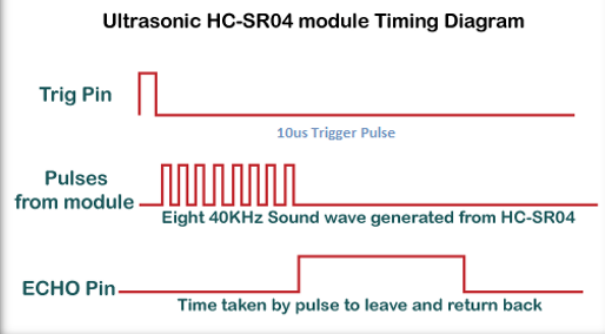
Fig 3.6: Working Principle

- Speed of Sound = 343 m/sec = 34300 cm/sec
- Distance Detected = (speed $\times$ time) / 2
= (34300 $\times$ time) / 2
= 17150 $\times$ time

- Distance Detected = (17150 $\times$ time) cm/sec

Here,

- time = Time duration when echo pin is high
= echo_pulse_end - echo_pulse_start



- Set the trig pin **high** for some time(3 to 100 us)
- As soon as trig pin is low, the sensor **send pulses** and **set echo pin to high**
- When the sensor gets the reflected pulses, it set the echo pin to low.
- We need to measure time for which echo pin was high.

Fig 3.7: Distance Calculation

3.7 Camera

The Raspberry Pi Camera is a camera module specifically designed to be used with Raspberry Pi single-board computers. It provides an easy and affordable way to capture images and videos directly from a Raspberry Pi, expanding its capabilities for various projects that involve imaging and computer vision.

Here are some key aspects of the Raspberry Pi Camera:

- **Hardware:** The Raspberry Pi Camera module consists of a small PCB (Printed Circuit Board) with a camera sensor, lens, and a flexible ribbon cable for connecting to the Raspberry Pi's camera connector. The camera sensor resolution and lens options may vary depending on the specific model of the camera module.
- **Image and Video Capture:** The camera module allows you to capture still images and record videos in different resolutions and formats. It supports a range of settings such as exposure, white balance, image effects, and more, allowing you to customize the capture parameters according to your needs.
- **High-Quality Imaging:** The Raspberry Pi Camera is capable of capturing high-quality images and videos, offering options for both color and monochrome (NoIR) versions. It provides good image sensor sensitivity and dynamic range, allowing for clear and detailed imaging in various lighting conditions.
- **Interfacing with Raspberry Pi:** The camera module connects to the Raspberry Pi via the dedicated camera interface connector, which provides a high-speed data transfer link between the camera module and the Raspberry Pi's processor. This interface ensures efficient and reliable communication between the camera and the board.
- **Software Integration:** The Raspberry Pi Camera is supported by the Raspberry Pi's official operating system, Raspberry Pi OS. It has a dedicated camera module interface that enables you to access the camera's functionality through software. Raspberry Pi OS provides libraries and APIs (such as the picamera Python library) that make it easy to capture images and control the camera settings programmatically.
- **Applications:** The Raspberry Pi Camera module is widely used in various applications, including photography, time-lapse photography, security systems, surveillance, video streaming, computer vision projects, robotics, and more. Its versatility and affordable price make it a popular choice for hobbyists, students, and professionals alike.

The Raspberry Pi Camera module offers a convenient and cost-effective way to add imaging capabilities to Raspberry Pi projects. With its integration with the Raspberry Pi ecosystem, it provides easy access to image and video capture functionalities, allowing users to explore a wide range of applications involving visual data.



Fig 3.8: Raspberry Pi Camera

3.8 Eyeglass

We used a simple eyeglass with broad side frame, so that raspberrypi can be attached to it. All the tools like raspberrypi, camera, ultrasonic sensor are attached to it.



Fig 3.9: Eyeglass

3.9 Wires

Wires are used to connect all the equipments together.



Fig 3.10: Wires

3.10 Battery

Battery is used to power the raspberry pi.



Fig 3.11: Battery

3.11 Earphone

Earphone is used to give audio output. These are attached to the aux port of raspberrypi.



Fig 3.12: Earphone

CHAPTER 4

CONNECTIONS & CODE

Connections are made by connecting power supply, earphone, ultrasonic sensor and camera from raspberrypi.

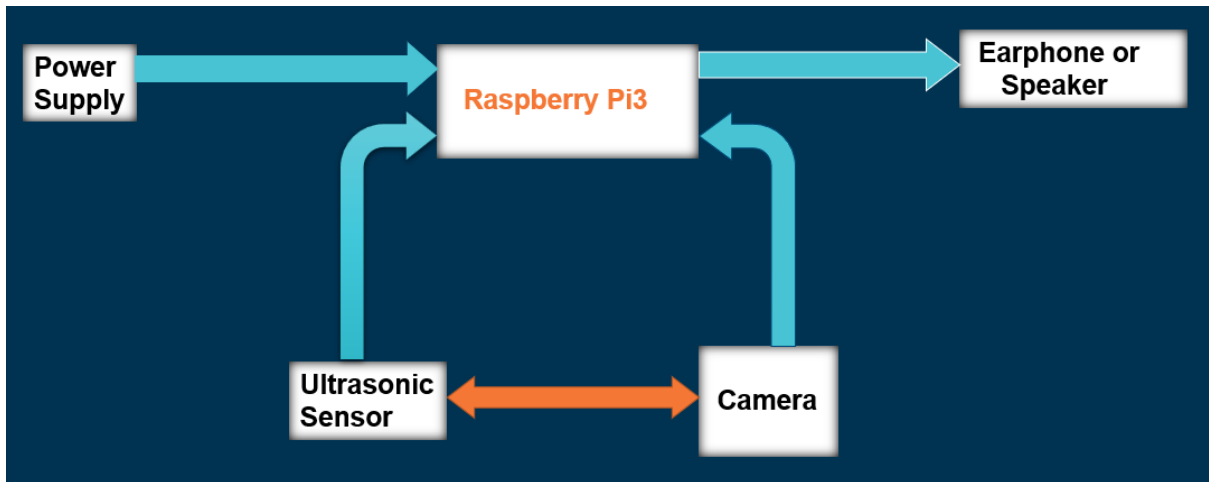


Fig 4.1: Block Diagram of Connection

4.1 Ultrasonic Sensor

Ultrasonic sensor is connected to the raspberrypi in the following way:

- Trig: 21 GPIO pin (40th Pin)
- Echo: 20 GPIO pin (38th Pin)
- VCC: 5V PWR Pin (4th Pin)
- GND: GND Pin (6th Pin)

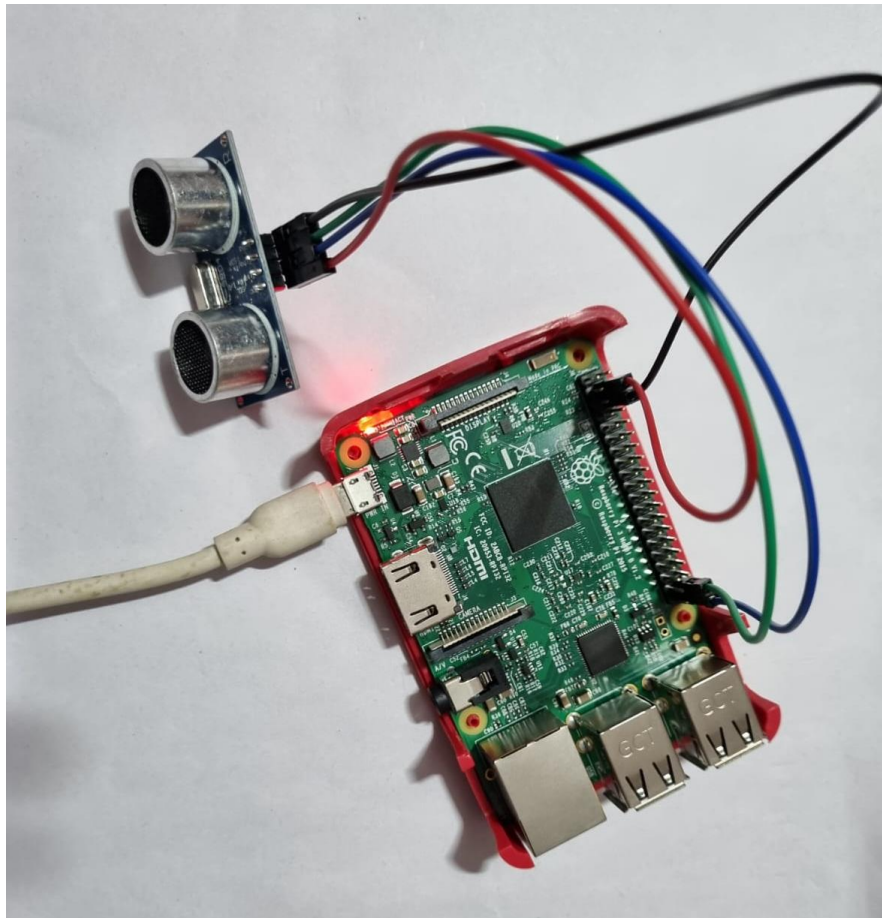


Fig 4.2: Ultrasonic sensor connected to Raspberry Pi

It is used to calculate distance of obstacle. It uses the following **code for distance calculation**:

```
import RPi.GPIO as GPIO
import time
from espeak import espeak

TRIG=21
ECHO=20
GPIO.setmode(GPIO.BCM)

while True:
    print("Distance measurement in progress")
    GPIO.setup(TRIG, GPIO.OUT)
```

```

GPIO.setup(ECHO, GPIO.IN)
GPIO.output(TRIG, False)
print("waiting for sensor to settle")
time.sleep(0.2)
GPIO.output(TRIG, True)
time.sleep(0.00001) # wait for 10 micro seconds
GPIO.output(TRIG, False)

while (GPIO.input(ECHO)==0): # wait till echo is low
    pulse_start=time.time()

while (GPIO.input(ECHO)==1): # wait till echo is high
    pulse_end=time.time()

pulse_duration=pulse_end-pulse_start
distance=pulse_duration*17150
print("Distance :",distance,"cm")

espeak.set_voice("f3")
espeak.synth("Obstacle is nearby")
espeak.synth(str(round (distance,1)))
espeak.synth("centimeters")

time.sleep(5)

```

4.2 Raspberry Pi Camera

It is connected to the camera port given on the raspberrypi. It captures images and pass it to our model to recognize the person.



Fig 4.3: Camera connected to Raspberry Pi

Code for face-recognition:

```
# -*- coding: utf-8 -*-
import face_recognition
import cv2
import numpy as np
from espeak import espeak

video_capture = cv2.VideoCapture(0)
```

```

#list of known family members for
# Family member 1
minhaaz_image = face_recognition.load_image_file("Minhaaz_pic.jpg")
minhaaz_face_encoding = face_recognition.face_encodings(minhaaz_image)[0]

gullu_image=face_recognition.load_image_file("Gullu.png")
gullu_face_encoding = face_recognition.face_encodings(gullu_image)[0]

shahabuddin_image=face_recognition.load_image_file("Shahabuddin.jpg")
shahabuddin_face_encoding =
face_recognition.face_encodings(shahabuddin_image)[0]

garv_image=face_recognition.load_image_file("Garv.jpg")
garv_face_encoding = face_recognition.face_encodings(garv_image)[0]

surya_image=face_recognition.load_image_file("Surya.jpg")
surya_face_encoding = face_recognition.face_encodings(surya_image)[0]

# Create arrays of known face encodings and their names
known_face_encodings = [
    minhaaz_face_encoding,
    gullu_face_encoding,
    shahabuddin_face_encoding,
    garv_face_encoding,
    surya_face_encoding
]

known_face_names = [
    "Minhaaz",
    "Gullu",
    "Shahabuddin",
    "Garv",
    "Surya"
]

# Initialize some variables
face_locations = []
face_encodings = []
face_names = []
process_this_frame = True

while True:
    # Grab a single frame of video
    ret, frame = video_capture.read()

    # Resize frame of video to 1/4 size for faster face recognition processing
    small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)

```

```

    # Convert the image from BGR color (which OpenCV uses) to RGB color (which
face_recognition uses)
    rgb_small_frame = small_frame[:, :, ::-1]

    # Only process every other frame of video to save time
    if process_this_frame:
        # Find all the faces and face encodings in the current frame of video
        face_locations = face_recognition.face_locations(rgb_small_frame)
        face_encodings = face_recognition.face_encodings(rgb_small_frame,
face_locations)

        face_names = []
        for face_encoding in face_encodings:
            # See if the face is a match for the known face(s)
            matches = face_recognition.compare_faces(known_face_encodings,
face_encoding)
            name = "Unknown Person"

            face_distances =
face_recognition.face_distance(known_face_encodings, face_encoding)
            best_match_index = np.argmin(face_distances)
            if matches[best_match_index]:
                name = known_face_names[best_match_index]
                espeak.set_voice("En")
                espeak.set_voice("f3")
                espeak.synth("Person name is ")

            face_names.append(name)
            espeak.synth(name)

        process_this_frame = not process_this_frame

    # Display the results
    for (top, right, bottom, left), name in zip(face_locations, face_names):
        # Scale back up face locations since the frame we detected in was
scaled to 1/4 size
        top *= 4
        right *= 4
        bottom *= 4
        left *= 4

        # Draw a box around the face
        cv2.rectangle(frame, (left, top), (right, bottom), (0, 0, 255), 2)

        # Draw a label with a name below the face
        cv2.rectangle(frame, (left, bottom - 35), (right, bottom), (0, 0,
255), cv2.FILLED)

```

```

        font = cv2.FONT_HERSHEY_DUPLEX
        cv2.putText(frame, name, (left + 6, bottom - 6), font, 1.0, (255, 255,
255), 1)

    # Display the resulting image
    cv2.imshow('Video', frame)

    # Hit 'q' on the keyboard to quit!
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Release handle to the webcam
video_capture.release()
cv2.destroyAllWindows()

```

4.3 Full Connection

All testing individual component, we combined both so that they can work together.

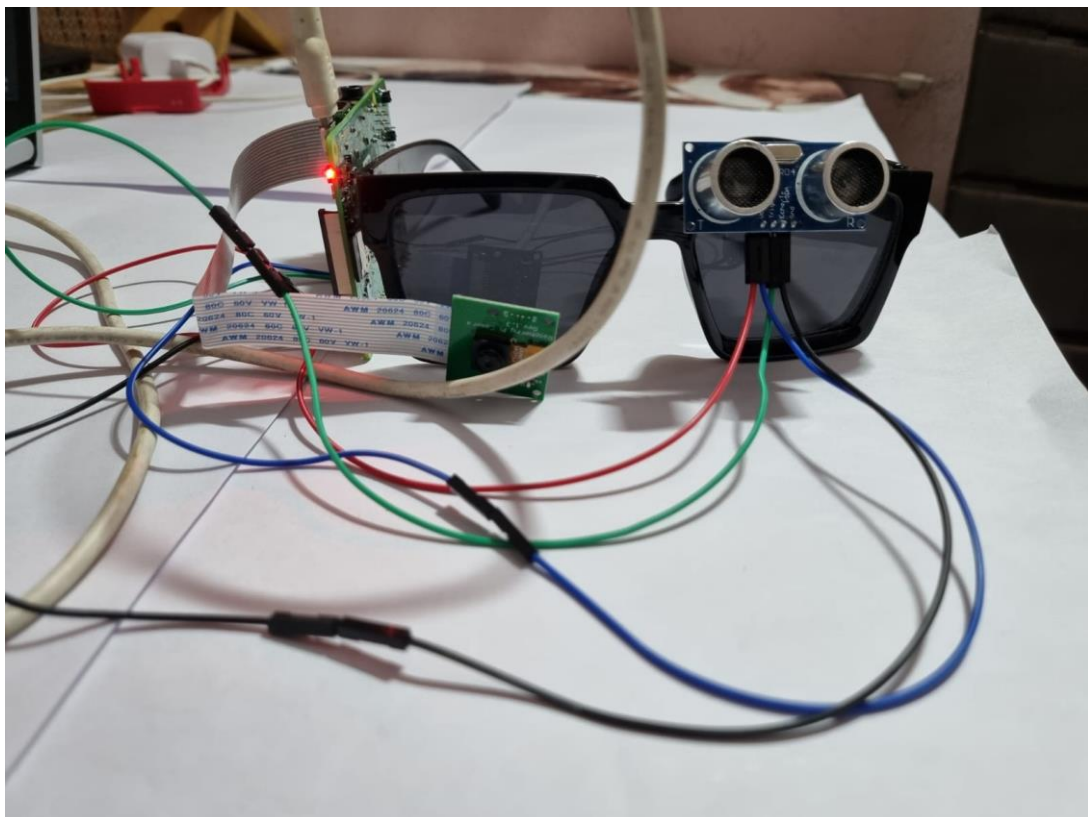


Fig 4.4: Full connection on Eyeglass

Code for combined connection:

```
import RPi.GPIO as GPIO
import time
from espeak import espeak
import face_recognition
import cv2
import numpy as np

TRIG=21
ECHO=20
GPIO.setmode(GPIO.BCM)

video_capture = cv2.VideoCapture(0)

#list of known family members for
# Family member 1
minhaaz_image = face_recognition.load_image_file("Minhaaz_pic.jpg")
minhaaz_face_encoding = face_recognition.face_encodings(minhaaz_image)[0]

gullu_image=face_recognition.load_image_file("Gullu.png")
gullu_face_encoding = face_recognition.face_encodings(gullu_image)[0]

# Create arrays of known face encodings and their names
known_face_encodings = [
    minhaaz_face_encoding,
    gullu_face_encoding
]

known_face_names = [
    "Minhaaz",
    "Gullu"
]

# Initialize some variables
face_locations = []
face_encodings = []
face_names = []
process_this_frame = True

while True:
    print("Distance measurement in progress")
    GPIO.setup(TRIG, GPIO.OUT)
    GPIO.setup(ECHO, GPIO.IN)
    GPIO.output(TRIG, False)
```

```

print("waiting for sensor to settle")
time.sleep(0.2)
GPIO.output(TRIG, True)
time.sleep(0.00001) # wait for 10 micro seconds
GPIO.output(TRIG, False)

while (GPIO.input(ECHO)==0): # wait till echo is low
    pulse_start=time.time()

while (GPIO.input(ECHO)==1): # wait till echo is high
    pulse_end=time.time()

pulse_duration=pulse_end-pulse_start
distance=pulse_duration*17150
print("Distance :",distance,"cm")

espeak.set_voice("f3")
espeak.synth("Obstacle is nearby")
espeak.synth(str(round (distance,1)))
espeak.synth("centimeters")

time.sleep(1)

# Grab a single frame of video
ret, frame = video_capture.read()

# Resize frame of video to 1/4 size for faster face recognition processing
small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)

# Convert the image from BGR color (which OpenCV uses) to RGB color (which
face_recognition uses)
rgb_small_frame = small_frame[:, :, :-1]

# Only process every other frame of video to save time
if process_this_frame:
    # Find all the faces and face encodings in the current frame of video
    face_locations = face_recognition.face_locations(rgb_small_frame)
    face_encodings = face_recognition.face_encodings(rgb_small_frame,
face_locations)

    face_names = []
    for face_encoding in face_encodings:
        # See if the face is a match for the known face(s)
        matches = face_recognition.compare_faces(known_face_encodings,
face_encoding)
        name = "Unknown Person"

```

```

        # # If a match was found in known_face_encodings, just use the
first one.
        # if True in matches:
        #     first_match_index = matches.index(True)
        #     name = known_face_names[first_match_index]

        # Or instead, use the known face with the smallest distance to the
new face
        face_distances =
face_recognition.face_distance(known_face_encodings, face_encoding)
        best_match_index = np.argmin(face_distances)
        if matches[best_match_index]:
            name = known_face_names[best_match_index]
            espeak.set_voice("En")
            espeak.set_voice("f3")
            espeak.synth("Person name is ")

        face_names.append(name)
        espeak.synth(name)

process_this_frame = not process_this_frame

# Display the results
for (top, right, bottom, left), name in zip(face_locations, face_names):
    # Scale back up face locations since the frame we detected in was
scaled to 1/4 size
    top *= 4
    right *= 4
    bottom *= 4
    left *= 4

    # Draw a box around the face
    cv2.rectangle(frame, (left, top), (right, bottom), (0, 0, 255), 2)

    # Draw a label with a name below the face
    cv2.rectangle(frame, (left, bottom - 35), (right, bottom), (0, 0,
255), cv2.FILLED)
    font = cv2.FONT_HERSHEY_DUPLEX
    cv2.putText(frame, name, (left + 6, bottom - 6), font, 1.0, (255, 255,
255), 1)

# Display the resulting image
cv2.imshow('Video', frame)

# Hit 'q' on the keyboard to quit!
if cv2.waitKey(1) & 0xFF == ord('q'):

```

```
        break

#    time.sleep(5)

# Release handle to the webcam
video_capture.release()
cv2.destroyAllWindows()
```

CHAPTER 5

WORKING

5.1 Steps

Project workflow is:

- Ultrasonic sensor and camera module continuously tries to detect objects.
- Once, an object is detected, then ultrasonic sensor calculates its distance.
- Camera checks, if it's a human or not.
- If it's a human, then it tries to match face with already saved faces.
- If a match is found, then it speaks name of the person using espeak module.
- If no match for the person is found, then it speaks unknown person.
- This process keeps on repeating.

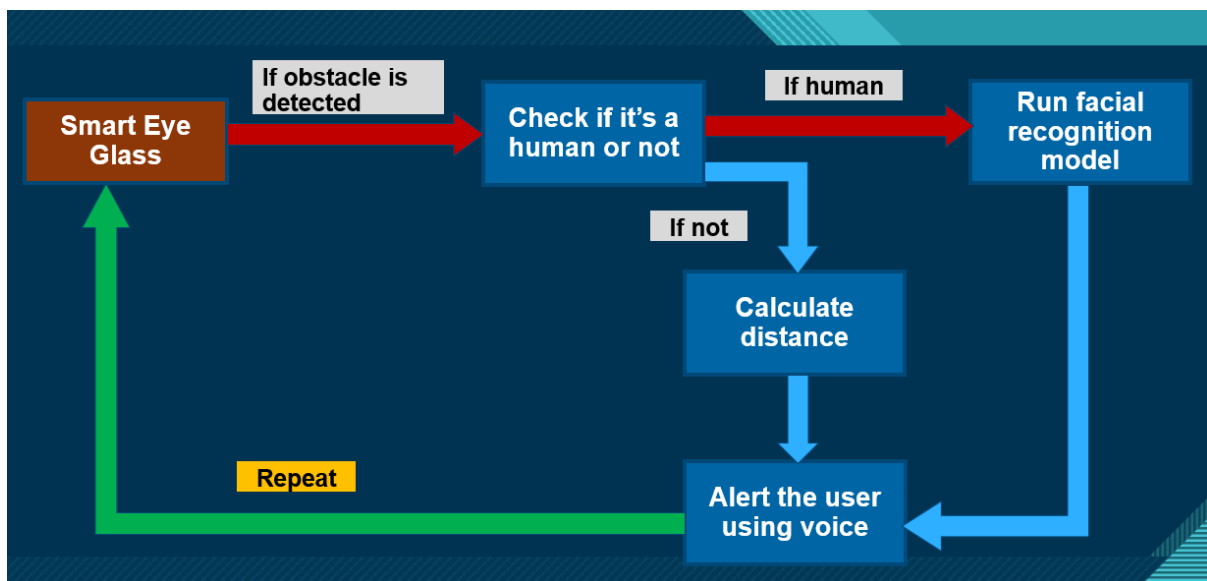


Fig 5.1: Work Flow

CHAPTER 6

RESULT

- Model is working fine for object detection. It detects any object, and calculates its distance accurately.
- It is also able to detect humans and recognize them with an accuracy of more than **90%**.
- It gives correct audio output with person name and object distance.

CHAPTER 7

LIMITATIONS

- Its most of the limitations are associated with limitations of ultrasonic sensor.
- It is unable to detect object if waves are not reflected back at the sensor.
- It can happen in the following cases:
 - If the object is far away (i.e., Out of range of ultrasonic sensor).
 - If the object is too small.
 - If the object is too soft.
 - If the flat object is not facing sensor (in this case, waves will reflect to other direction).

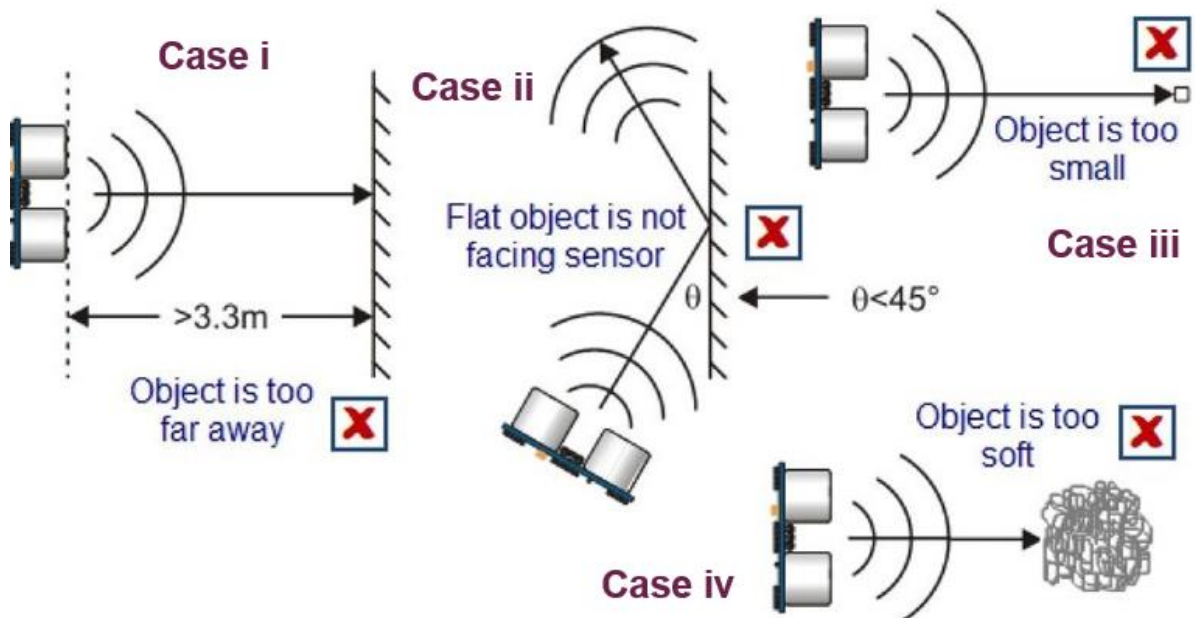


Fig 7.1: Limitations of Ultrasonic sensor

- It detects obstacles which are coming from only in front of ultrasonic sensor. If obstacle is coming from other directions, then it will not be able to detect them.
- It struggles to recognize faces if there is no proper lighting.
- It is only able to recognize those faces which are already saved in memory.

CHAPTER 8

SOLUTIONS OF LIMITATIONS

8.1 Ultrasonic Sensor Problem

- Object distance problem can be solved using an ultrasonic sensor of high range.
- Object coming from sideways can be detected using multiple ultrasonic sensors, which are mounted in different directions on the eyeglass.

8.2 Camera Problem

- Low light problem can be solved using high quality camera, which can take good photos in low light.
- We can also use a flashlight, which can be attached to the eyeglass for light.
- A program can be written where new faces can be added on the server.

REFERENCES

- [1] <https://www.who.int/news-room/fact-sheets/detail/blindness-andvisualimpairment#:~:text=Globally%2C%20at%20least%202.2%20billi on,near%20or%20distance%20vision%20impairment>
- [2] Ultrasonic Smart Spectacles for Visually Impaired and Blind People
Authors: L. Shashitha (Department of Electronics and Communication Engineering, Institute of Aeronautical Engineering College Dundigal, Hyderabad, Telangana) P. Ashok Babu (Department of Electronics and Communication Engineering, Institute of Aeronautical Engineering College Dundigal, Hyderabad, Telangana)
Published in: 2021 3rd International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)
Date of Conference: 17-18 December 2021 Date Added to IEEE Xplore: 09 March 2022 Electronic
ISBN:978-1-6654-3811-7 Print on Demand(PoD)
ISBN:978-1-6654-3812-4
INSPEC Accession Number: 21666268
DOI: 10.1109/ICAC3N53548.2021.9725747
Publisher: IEEE Conference
Location: Greater Noida, India